

Title

Course Name: DLMDSPWP01 – Programming with Python

Program: Master's in computer science

Date: 5th August 2022

Submitted by Wycliffe Bahati

Matriculation Number: 92014201

Tutor: Cosmina Croitoru

Tasks for Course: Parallel Computing in Python

Table of contents

Contents

Title	i
Background	3
Parallel Computing Techniques in Python	3
Threading	3
Asynchronous I/O	5
Multiprocessing	6
Charm4py	7
Mpi4py	8
Pyro5	8
Performance Analysis	8
Empirical measurement	8
Asymptotic analysis	9
Challenges and Limitations	11
GIL	11
Latency	11
Speed up of performance.	11
Data race	11
Deadlock	12
Complexity	12
Future Directions	12
Conclusion	12
Appendices	13
Programming Task	13
Code	13
Bibliography	19

List of abbreviations (if necessary)

CPU – Central Processing Unit

GIL – Global Interpreter Lock

CHAM4PY – CHAM++ for python

MPI4PY – MPI for Python

API – Application Program Interface

Figures and diagrams

Introduction

Traditionally software has been written to be executed in a sequential manner. This is common practice for programmers because its simple whoever it has limitations. Following the limitations in the 21st century to design faster processors, some vendors have designed multicore CPUs. This involved design which couple of CPUs on a single chip. This brings opportunities for programmers to take advantage of the extra CPU's to design their programs to run tasks in parallel.(Andrade et al., 2023, p. 3)

Parallel computing is a type of computing where a program can be divided into subprograms or tasks and can be executed simultaneously. Parallel computing adds speed of execution on the programs as compared to sequential program execution which is entirely dependent on the speed of the processor. (*Sequential vs. Parallel Computing*, n.d.)

The operating system manages hardware and software resources as well as scheduling how programs run on the computer. These programs are run as processes with a process id. A program can be invoked several times to run as different processes. The work of an operating system is to multitask these processes so they can share time on the available on the available processors. Processes are identified by the process identification. A process has allocated resources such as memory and cannot access memory belonging to another process which makes inter process communication very difficult. Inter process communication can be made through Inter Process communication and via sockets and pipes, explicitly shared memory, or remote procedure calls. However, this is made easier if we launch intercommunication within the program. This is where we find the use of threads.

Each process can have several threads. The threads share the same memory address space. This makes it easier for the threads to communicate amongst each other.

To illustrate parallel computing we look at a simple task below.

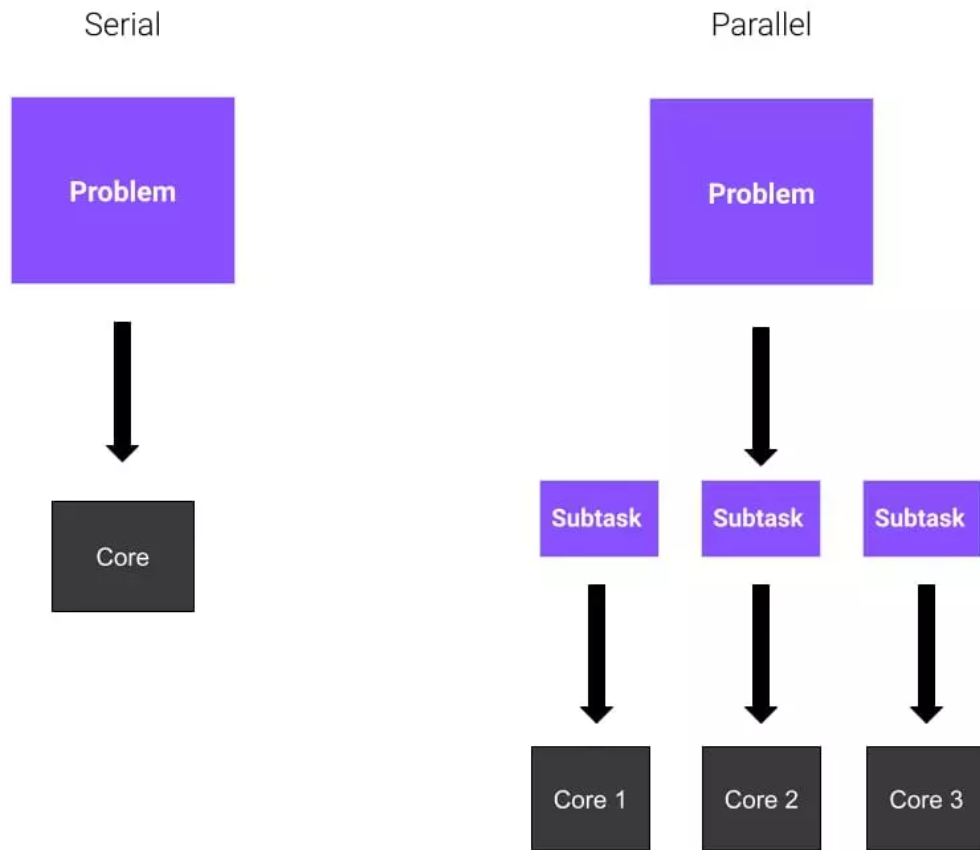


Fig 1.1 source (*A Guide to Python Multiprocessing and Parallel Programming — SitePoint, 2022*)

Given parallel computing a program will generally execute faster than the sequential counterpart if the hardware supports parallel computing. However, there are cases where the computation requires shared resources such as an array that is stored in memory. Such cases might not result in exponential speed up of the computation. Another case where we might not get an advantage of parallel computing is when the two subtasks of a program have different lengths of time needed for execution. In this case the task that takes a shorter time must wait for some time for the longer task to complete. ("Parallel Computing," 2024)

To support parallel computing efficiently we need the language and libraries, operating system, virtual machine, and operating system to support parallel computing.

In this assignment we will explore the python language and the available tools that can be used to write parallel programming that can take advantage of multi-core environments in modern computers.

Background

Before the 20th century parallel computing was mainly restricted to specialized tasks such as weather focusing and military applications. This led to the development of parallel hardware. Quad-core processors are now standard in personal computers. This has made it possible for programmers to develop programs that can be structured to be computed in parallel. It is predicted that the development of processors follows Moore's law it can be predicted that the number of cores per processor will double every 18–24 months. Many programs are written to execute in a serial form. Programmers need to adapt to parallel computing to take advantage of the multi-core processors. ("Parallel Computing," 2024)

Parallel Computing Techniques in Python

The most popular interpreter for python is the CPython. This interpreter implements the GIL for thread safe operations. The python interpreter (GIL) does not allow for threads to be executed in parallel. In a multithread architecture with multiple CPU's the GIL becomes an issue to be addressed. For programs that are CPU bound and spend most of their time in execution the GIL would affect the threads as they would execute in sequence. However, there is an opportunity with programs that depend a lot on I/O operations as they take time to execute. With this type of operations parallel programming could make a difference. To overcome this, we can use the python multiprocessing package to implement parallel processes. In this case each python thread will run on its own interpreter. (*Python Parallel Processing*, n.d.)

There are three main libraries in python that support parallel programming. Threading, AsyncIO and Multiprocessing. There are other frameworks that enable parallel programming in python which include Charm4py and Mpi4py (Fink et al., 2021, p. 2). In this chapter we will discuss these libraries and frameworks

Threading

The threading library allows us to execute multiple tasks in parallel. Though the GIL will only one process to execute at a time. Threads share memory and can use synchronization lock, acquire and release manage access to resources. Instead of waiting for a task to complete, the program can utilize the time to perform other tasks. To use threading, one must import the

threading function. Below is an example of a threading function that computes the square of several number using a thread.

```
import threading

def print_numbers():

    for i in range(10):
        print("This thread prints numbers: {}".format(i))

def print_letters():

    for letter in 'abcdefghij':
        print("This thread prints letters: {}".format(letter))

t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)

t1.start()
t2.start()

t1.join()
t2.join()
```

(Python Parallel Processing, n.d.)

This program starts 2 threads `print_numbers` and `print_letters`. Sometimes threads are run in parallel to each of each other. The output below

```
This thread prints numbers: 0
This thread prints numbers: 1This thread prints letters: a
This thread prints letters: b
This thread prints letters: c
This thread prints letters: d
This thread prints letters: e
This thread prints letters: f
This thread prints letters: g
This thread prints letters: h
This thread prints letters: i
This thread prints letters: j

This thread prints numbers: 2
This thread prints numbers: 3
This thread prints numbers: 4
```



```
This thread prints numbers: 5
This thread prints numbers: 6
This thread prints numbers: 7
This thread prints numbers: 8
This thread prints numbers: 9
```

We can see that only once the program was able to print in parallel. Most of the time Python will run them in sequence because of the GIL. However, if one of the functions require I/O input then we would have full advantages of running threading. We can introduce barriers that will force a thread to wait until the other thread is complete however in this case it does not add efficiency to our program. We will discuss more of this when we look at the challenges and limitations later. (*Threading — Thread-Based Parallelism*, n.d.)

Asynchronous I/O

Sometimes the program may have tasks that require I/O operations. Instead of the program waiting for input from the disk, database, or network it could go ahead and execute other tasks as it waits for the input. Thankfully python has included the `asyncio` module provides libraries and functions that enable the running of co-routines. Asynchronous routines can pause and allow other routines to run as they wait for the results of their operations. (Python, n.d.-a). Below is a sample program that will use utilizes the `asyncio` module

```
import asyncio
import aiohttp

async def fetch(session, url):
    async with session.get(url) as response:
        return await response.text()

async def main():
    urls = ['http://python.org', 'http://google.com', 'http://yahoo.com']
    tasks = []

    async with aiohttp.ClientSession() as session:
        for url in urls:
            tasks.append(fetch(session, url))

    pages = await asyncio.gather(*tasks)
```

```

        for url, page in zip(urls, pages):
            print(f'URL: {url}, Content Length: {len(page)}')

if __name__ == '__main__':
    asyncio.run(main())

```

(Python, n.d.-a)

In this program `asyncio.gather` is used to run the task to fetch the different urls in parallel.

Multiprocessing

The GIL implementation restricts python interpreter to executing one process at a time.

Multiprocessing library supports spawning of several processes. This means that every process can run on its own virtual environment. For programs that spend most of the time within the CPU. This can assist to achieve improved performance. However, the challenges come with the communication between the process as they do not share the same memory space. To utilize the multiprocessing library, we first need to import the multiprocessing module.

```

from multiprocessing import Process

```

Then we will replace the `threading` with `thread` module from the `multithreading` module with the `process` module in the multiprocessing module.

Below we illustrate the previous program but with multiprocessing. We can see that the process ID is different for the different functions (*Multiprocessing — Process-Based Parallelism*, n.d.)

```

from multiprocessing import Process
import os
def print_numbers():
    for i in range(3):
        print("This thread prints numbers: {}".format(i))
        print('process id:', os.getpid())

def print_letters():
    for letter in 'abcd':
        print("This thread prints letters: {}".format(letter))
        print('process id:', os.getpid())
if __name__ == '__main__':
    p1 = Process(target=print_numbers)
    p2 = Process(target=print_letters)

    p1.start()
    p2.start()

```

```
p1.join()
p2.join()
```

(*Python Parallel Processing*, n.d.)

The output is the output to include the process ID

```
this thread prints letters: a
process id: 9428
This thread prints letters: b
process id: 9428
This thread prints letters: c
process id: 9428
This thread prints letters: d
process id: 9428
This thread prints numbers: 0
process id: 25320
This thread prints numbers: 1
process id: 25320
This thread prints numbers: 2
process id: 25320
```

Apart from the existing libraries in python. We briefly describe different frameworks that have been developed to enhance parallel programming. Below we will look at 2 of such frameworks. Charm4py and Mpi4py

Charm4py

Charm4py is a parallel programming framework in python that is built on Charm++. According (*Charm++: Parallel Programming Framework*, n.d.) to “Charm++ is a parallel programming framework in C++ supported by an adaptive runtime system, which enhances user productivity and allows programs to run portably from small multicore computers (your laptop) to the largest supercomputers”

Cham4py is a parallel programming framework built on python language. A good parallel programming framework must achieve accessibility, performance, high level abstraction from hardware and software, portable, easy to integrate with existing software and achieve productivity. (*Charm++: Parallel Programming Framework*, n.d., p. 423). Charm4py is based on distributed migratable objects and has scalability meaning it can be deployed on laptops and supercomputers.

Charm4py make it easier to write and deploy parallel programming features. It is based on distributed objects that can run on different processors. This object can communicate with each other using remote object invocation and message passing. These objects are dynamic in nature and do not necessarily reside on one processor (*Charm++: Parallel Programming Framework*, n.d., p. 424). The distributed object are known as *Chares*

Mpi4py

When programs must run on different processors one of the challenges is how they communicate to pass variables, data, and functions. MPI4py is a message passing API developed from MPI which is a standardized application for message passing for inter process communication that originally was popular with C++ and Fortran.(Dalcin & Fang, 2021, p. 47)

Mpi4py takes advantage of python's pickle protocol which implements binary protocols for object serialization and deserialization.(*Pickle — Python Object Serialization*, n.d.)

Pyro5

Pyro5 enables building applications in which objects can talk to each other over the network. Its build using python and runs on different platforms (*Pyro - Python Remote Objects - 5.15 — Pyro 5.15 Documentation*, n.d.)

Performance Analysis

The main advantage of parallel computing is the performance advantage. We can measure performance by two methods. To measure performance we involve Time, resources and costs used to execute a task

Empirical measurement

We can measure the amount of time the program takes to execute using the inbuilt time module. An example of our previous asyncio program is illustrated below.

```
import asyncio
import aiohttp
import time

async def fetch(session, url):
    async with session.get(url) as response:
        return await response.text()

async def main():
    urls = ['http://python.org', 'http://google.com', 'http://yahoo.com']
    tasks = []

    async with aiohttp.ClientSession() as session:
```

```

    for url in urls:
        tasks.append(fetch(session, url))

    pages = await asyncio.gather(*tasks)

    for url, page in zip(urls, pages):
        print(f'URL: {url}, Content Length: {len(page)}')

if __name__ == '__main__':
    start_time = time.time()
    asyncio.run(main())
    end_time = time.time()

    print(f'Execution time: {end_time - start_time} seconds')

```

The output will tell us the time the program took to execute. With the output as below.

```

DES\AppData\Local\Temp\tempCodeRunnerFile.python"
URL: http://python.org, Content Length: 50671
URL: http://google.com, Content Length: 19261
URL: http://yahoo.com, Content Length: 1763236
Execution time: 8.838724374771118 seconds

```

Asymptotic analysis

We use asymptotic analysis to measure performance when we have large inputs and more parallel hardware. In asymptotic analysis we try to determine the fastest and slowest execution time for a program. The simplest way to perform Asymptotic analysis is to perform time measurement one access on different data structures and measurement how long it takes to perform operations on the data structures varying the inputs. However, this is not very practical, especially when measuring the performance of different machines. The example python code below inserts an element at the beginning of an array.

```

import time
import random

def insert_at_beginning(arr, element):
    start_time = time.time()
    arr.insert(0, element)
    end_time = time.time()
    return end_time - start_time

```

```

# Create an array of size 10000 filled with random integers
array_size = 100000
my_array = [random.randint(1, 100) for _ in range(array_size)]

# New element to insert at the beginning
new_element = 0

# Measure the time taken to insert the element at the beginning
elapsed_time = insert_at_beginning(my_array, new_element)
print("Time taken:", elapsed_time, "seconds")

```

As we vary the input the time increases as output below

```

DES\AppData\Local\Temp\tempCodeRunnerFile.python"
Time taken: 0.0 seconds to insert 10000 array size

[Done] exited with code=0 in 0.134 seconds

[Running] python -u
"C:\Users\ADMIN~1\DES\AppData\Local\Temp\tempCodeRunnerFile.python"
Time taken: 0.0 seconds to insert 100000 array size

[Done] exited with code=0 in 0.162 seconds

[Running] python -u
"C:\Users\ADMIN~1\DES\AppData\Local\Temp\tempCodeRunnerFile.python"
Time taken: 0.0 seconds to insert 1000000 array size

[Done] exited with code=0 in 0.439 seconds

```

** The results obtained from a machine running other programs and might change depending on the machine

Clearly from our experiment above the time is dependent on the input. So for input size n we can denote $f(n)$ to denote the time complexity

some mathematical notations that can be used to compute the time complexity are

Big O notation: to represent the longest time it takes for an algorithm to execute.

Omega notation: to represent lower bound of an algorithm to execute.

Theta notation: to represent the minimum and maximum time it takes for an algorithm to execute.

(Parallel Algorithm - Analysis, n.d.)

Challenges and Limitations

Processing in parallel has its advantages and disadvantages. Processing the program in parallel introduces challenges that need to be addressed. We look at some of the challenges in this section.

GIL

The python GIL implementation only allows one thread to execute at a time. This would translate to only one thread executing at the same time even if there are multiple threads. The default interpreter for python is CPython which is written in C and python. The GIL was implemented to provide thread safe operations and memory management. (Python, n.d.-b) There are however other alternative interpreters such as Jython and Iron Python that one can use.

Latency

In the case where we have the algorithm distributed in different networks. This introduces latency via the inter process communication. This would result in slower execution time given access to local memory is always faster than the network latency.

Speed up of performance.

According to Amdahl's law "the overall performance improvement gained by optimizing a single part of a system is limited by the fraction of time that the improved part is actually used" ("Amdahl's Law," 2024). There are parts of the algorithm that cannot be made parallel and have to be executed sequentially. The proportion of algorithm that is sequential must be limited in proportion to achieve any speed up in parallelizing the algorithm.

Data race

Data race occurs when several threads try to access the same shared memory location therefore modifying shared memory. This can result in program inconsistency. One of the methods that can be used to solve this is using mutual locks to protect the areas of the program that have access to the shared resource.

Deadlock

Deadlock occurs in parallel programming when a process cannot proceed as it waits for a locked resource by another process including itself. This is a possible scenario in parallel programming. The circular wait can lead to a deadlock.

Complexity

To manage a program that is running on parallel code requires more consideration as compared to the serial programs. Data race, inter process communication, resource management are some of the challenges making this complex.

Future Directions

There is need for current and future developers to be more Enlightened of the power of parallel computing. Factoring parallel computing in their tasks will increase performance especially when processing large amounts of data that is now common with machine learning and data science.

There is a lot of debate about how to improve the common python interpreter to accommodate parallel processing. This conversation is likely to mature into a more accommodating approach that would not compromise the simplicity and portability of the python programming language.(Yegulalp, 2023)

Current programming in python has several inbuilt libraries and frameworks that simplify parallel programming. These are likely to grow and become enhanced and we are likely to see more frameworks coming up especially to accommodate cloud computing.

Conclusion

Python is a powerful programming language with an elaborate data structure and numerous libraries. On top of the libraries, we have different frameworks that make parallel programming simpler. In this paper we have discussed the different techniques of parallel programming with python including threading, asyncio and multiprocessing. We also looked at a few frameworks for parallel computing including Charm4py, MPI4py and Pyro5.

We further looked at methods we can use to analyse performance such as the empirical analysis method and the asymptotic analysis method. Finally, we analyzed the challenges that are faced when developing algorithms to utilize the parallel features. There is a lot of promising future for parallel computing in python and we are bound to see more frameworks and changes in the default python interpreter to accommodate more diverse parallel computing.

Appendices

Programming Task

Code

```
import sqlite3
import csv
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

class create_database:
    def __init__(self, db_name):
        self.conn = sqlite3.connect(db_name)
        self.cursor = self.conn.cursor()
    def create_tables(self):
        conn = sqlite3.connect('dlmdspwp02.db')
        cursor = conn.cursor()
        # Define the SQL query to create the table
        create_table_train = '''
CREATE TABLE IF NOT EXISTS train_dataset (x FLOAT,{});
''.format(', '.join([f'y{i} FLOAT' for i in range(1, 5)])) # Define
columns y1 to y4
create_table_ideal = '''
CREATE TABLE IF NOT EXISTS ideal_dataset (x FLOAT,{});
''.format(', '.join([f'y{i} FLOAT' for i in range(1, 51)])) # Define
columns y1 to y5
create_table_mapping = '''
CREATE TABLE IF NOT EXISTS mappings (
    xtestf FLOAT,
    ytestf FLOAT,
    deltay FLOAT,
    chosenf TEXT
)'''
        # Execute the SQL query to create the tables
        #cursor.execute("DROP TABLE IF EXISTS train_dataset")
        cursor.execute(create_table_train)
        cursor.execute(create_table_ideal)
        cursor.execute( create_table_mapping)
        conn.commit()
        # Close the connection
        conn.close()
def readcsv_insertinto_traindataset():
    conn = sqlite3.connect('dlmdspwp02.db')
    cursor = conn.cursor()
    with open('train.csv', 'r') as file:
        reader = csv.reader(file)
```

```

        next(reader) # Skip header
    for row in reader:
        x = float(row[0])
        y1 = float(row[1])
        y2 = float(row[2])
        y3 = float(row[3])
        y4 = float(row[4])
        cursor.execute("INSERT INTO train_dataset (x, y1, y2, y3, y4) VALUES
(?, ?, ?, ?, ?)", (x, y1, y2, y3, y4))
    # Commit the transaction
    conn.commit()
    # Close the connection
    conn.close()
# insert data into the real dataset
def readcsv_insertinto_ideal_dataset():
    conn = sqlite3.connect('dlmdspwp02.db')
    cursor = conn.cursor()
    with open('ideal.csv', 'r') as file:
        reader = csv.reader(file)
        next(reader) # Skip header
        for row in reader:
            x = float(row[0])
            y_values = [float(val) for val in row[1:]]
            placeholders = ', '.join(['?'] for _ in range(1, 51))
            insert_query = "INSERT INTO ideal_dataset (x, {}) VALUES (?,
{}).format(', '.join(['y{}'.format(i) for i in range(1, 51)]), placeholders)
            cursor.execute(insert_query, [x] + y_values)
        # Commit the transaction
        conn.commit()
        # Close the connection
        conn.close()
''' This Functions computes the y deviations on the training set and stores them
in an array train_ls '''
def compute_least_square_train():
    conn = sqlite3.connect('dlmdspwp02.db') # Connect to our database
    # Read contents of the table into a pandas DataFrame
    df = pd.read_sql_query("SELECT * FROM train_dataset", conn)
    number_of_columns= len(df.columns)
    #print(number_of_columns)
    # Close the connection
    conn.close()
    for t in range (1,number_of_columns):
        # Coomputing X and Y
        X = df['x'].values
        Y = df['y' + str(t)]

```

```

        # Total number of values
        n = len(X)
        # Mean of Yt
        mean_y = np.mean(Y)
        train_maxdev.append(np.max(Y))
        sum_of_devsq = 0
        denom = 0
        # Total number of values
        n = len(X)
        for i in range(n):
            sum_of_devsq += (Y[i] - mean_y) ** 2
            train_ls.append(sum_of_devsq)
def compute_least_square_ideal():
    conn = sqlite3.connect('dlmdspwp02.db')
    # Read contents of the table into a pandas DataFrame
    df = pd.read_sql_query("SELECT * FROM ideal_dataset", conn)
    number_of_columns= len(df.columns)
    # print(df)
    # Close the connection
    conn.close()
    for t in range (1,number_of_columns):
        # Computing Y
        X = df['x'].values
        Y = df['y' + str(t)]
        # Total number of values
        n = len(X)
        # Mean of Yt
        mean_y = np.mean(Y)
        ideal_chosen_function_max.append(np.max(Y))
        sum_of_devsq = 0
        denom = 0
        # Total number of values
        n = len(X)
        for i in range(n):
            sum_of_devsq += (Y[i] - mean_y) ** 2
            ideal_ls.append(sum_of_devsq)
def find_min_difference(array1, array2):
    min_diff = float('inf') # Initialize min_diff to infinity
    min_indices = 0 # Initialize min_indices to an invalid pair
    # Iterate over all pairs of elements
    for i in range(len(array1)):
        min_diff = float('inf') # Initialize min_diff to infinity
        min_indices = 0 # Initialize min_indices to an invalid pair
        for j in range(len(array2)):

```

```

        # Calculate the absolute difference between the current pair of
elements
        diff = abs(array1[i] - array2[j])
        # If this difference is less than the current minimum, update
min_diff and min_indices
        if diff < min_diff:
            min_diff = diff
            min_indices = j
        chosen_elements.append(min_indices)
        #print(min_diff,min_indices)
        #print("chosen function for
y"+str(i+1),"is",array2[(chosen_elements[i])],min_indices)
        return chosen_elements

def findmaximumdeviation(firstarray,secondarray):
    array1 = firstarray
    array2 = secondarray

    # Calculate the absolute differences between corresponding elements
    absolute_differences = np.abs(array1 - array2)
    return absolute_differences

def map_idealfunction_to_testdataset(chosen_in):
    x_values = []
    y_values = []
    k=0
    ideal_f=0
    conn = sqlite3.connect('dlmdspwp02.db')
    cursor = conn.cursor()
    df = pd.read_sql_query("SELECT * FROM ideal_dataset", conn)
    df_1 = pd.read_sql_query("SELECT * FROM train_dataset", conn)
    cursor.execute("DROP TABLE IF EXISTS mappings ")
    create_table_mapping = '''CREATE TABLE IF NOT EXISTS mappings (xtestf
FLOAT,ytestf FLOAT,deltay FLOAT,chosenf TEXT)'''
    cursor.execute(create_table_mapping)
    number_of_columns= len(df.columns)
    train_Y = df_1["y1"]
    ideal_Y = df['y'+str(chosen_in[0])]
    #print(ideal_Y)
    # Open the CSV file and read it line by line
    with open('test.csv', 'r') as file:
        reader = csv.reader(file)
        next(reader)
        for row in reader:
            # First column is x and the second column is y
            x_values=(float(row[0]))

```

```

        y_values=(float(row[1]))
        for t in range (len(chosen_in)):
            train_Y = df_1['y'+str(t+1)]
            ideal_Y = df['y'+str(chosen_in[t])]
            #Calculate the deviation
            deviations = np.abs(train_Y - ideal_Y)
            #calculate maximum deviation
            max_deviation = np.max(deviations)
            if max_deviation < y_values**2:
                print (x_values,y_values**2,
y_values**2,'y'+str(chosen_in[t]))
                cursor.execute("INSERT INTO mappings (xtestf, ytestf,
deltay, chosenf) VALUES (?, ?, ?, ?)", (x_values, y_values,
y_values**2,'y'+str(chosen_in[t])))
                conn.commit()

''' Global used variables'''
train_ls=[]
train_maxdev=[]
ideal_ls=[]
ideal_chosen_function_max=[]
chosenidealset=[]
chosen_indices=[]
chosen_elements=[]
train_lsq_chosen_ideal_ls=[]
'''End of Global variables'''
def main():
    db=create_database('dlmdspwp02.db')
    db.create_tables()
    #readcsv_insertinto_traindataset() Commented to avoid duplication
    #readcsv_insertinto_ideal_dataset() Commented to avoid duplication
    '''Compute the deviations for training and ideal set'''
    compute_least_square_train()
    compute_least_square_ideal()
    '''Call function to choose our 4 ideal functions'''
    chosen_indices=find_min_difference(train_ls, ideal_ls)
    ''' adjust the chosen ideal functions to reflect the correct indices'''
    for i in range (len(chosen_indices)):
        chosen_indices[i]+= 1
    for t in range (len(chosen_indices)):
        print("chosen function for trainnig set y"+str(t+1),"is
y"+str(chosen_indices[t]))
        '''call method to map our test dataset to the ideal chosen ideal functions'''
        map_idealfunction_to_testdataset(chosen_indices)
if __name__=='__main__':

```

```
main()
```

Bibliography

A Guide to Python Multiprocessing and Parallel Programming—SitePoint. (2022, August 4).

<https://www.sitepoint.com/python-multiprocessing-parallel-programming/>

Amdahl's law. (2024). In *Wikipedia*.

https://en.wikipedia.org/w/index.php?title=Amdahl%27s_law&oldid=1210992588

Andrade, G., Griebler, D., Santos, R., & Fernandes, L. G. (2023). A parallel programming assessment for stream processing applications on multi-core systems. *Computer Standards & Interfaces*, 84, 103691. <https://doi.org/10.1016/j.csi.2022.103691>

Charm++: Parallel Programming Framework. (n.d.). Retrieved March 18, 2024, from

<https://charmplusplus.org/>

Dalcin, L., & Fang, Y.-L. L. (2021). mpi4py: Status Update After 12 Years of Development. *Computing in Science & Engineering*, 23(4), 47–54. <https://doi.org/10.1109/MCSE.2021.3083216>

Fink, Z., Liu, S., Choi, J., Diener, M., & Kale, L. V. (2021). Performance Evaluation of Python Parallel Programming Models: Charm4Py and mpi4py. *2021 IEEE/ACM 6th International Workshop on Extreme Scale Programming Models and Middleware (ESPM2)*, 38–44.

<https://doi.org/10.1109/ESPM254806.2021.00010>

multiprocessing—Process-based parallelism. (n.d.). Python Documentation. Retrieved March 18, 2024, from <https://docs.python.org/3/library/multiprocessing.html>

Parallel Algorithm—Analysis. (n.d.). Retrieved March 19, 2024, from

https://www.tutorialspoint.com/parallel_algorithm/parallel_algorithm_analysis.htm

Parallel computing. (2024). In *Wikipedia*.

https://en.wikipedia.org/w/index.php?title=Parallel_computing&oldid=1210542869

pickle—Python object serialization. (n.d.). Python Documentation. Retrieved March 19, 2024, from

<https://docs.python.org/3/library/pickle.html>

Pyro—Python Remote Objects—5.15—Pyro 5.15 documentation. (n.d.). Retrieved March 20, 2024, from <https://pyro5.readthedocs.io/en/latest/>

Python Parallel Processing: 3 Methods with Examples. (n.d.). Retrieved March 18, 2024, from <https://www.run.ai/guides/deep-learning-for-computer-vision/python-parallel-processing>

Python, R. (n.d.-a). *Async IO in Python: A Complete Walkthrough – Real Python.* Retrieved March 10, 2024, from <https://realpython.com/async-io-python/>

Python, R. (n.d.-b). *What Is the Python Global Interpreter Lock (GIL)? – Real Python.* Retrieved March 20, 2024, from <https://realpython.com/python-gil/>

Sequential vs. Parallel computing. (n.d.). Retrieved March 7, 2024, from <https://www.linkedin.com/learning/python-parallel-and-concurrent-programming-part-1/sequential-vs-parallel-computing>

threading—Thread-based parallelism. (n.d.). Python Documentation. Retrieved March 10, 2024, from <https://docs.python.org/3/library/threading.html>

Yegulalp, S. (2023, August 4). *Python moves to remove the GIL and boost concurrency.* InfoWorld. <https://www.infoworld.com/article/3704248/python-moves-to-remove-the-gil-and-boost-concurrency.html>

